CNCF JORDAN

YAML

FOR KUBERNETES

Understanding how Kubernetes resources are defined

Week 5

Special Session

TABLE OF INDEX

1

WHAT IS YAML IN KUBERNETES

CNCF JORDAN

What Is YAML in Kubernetes?

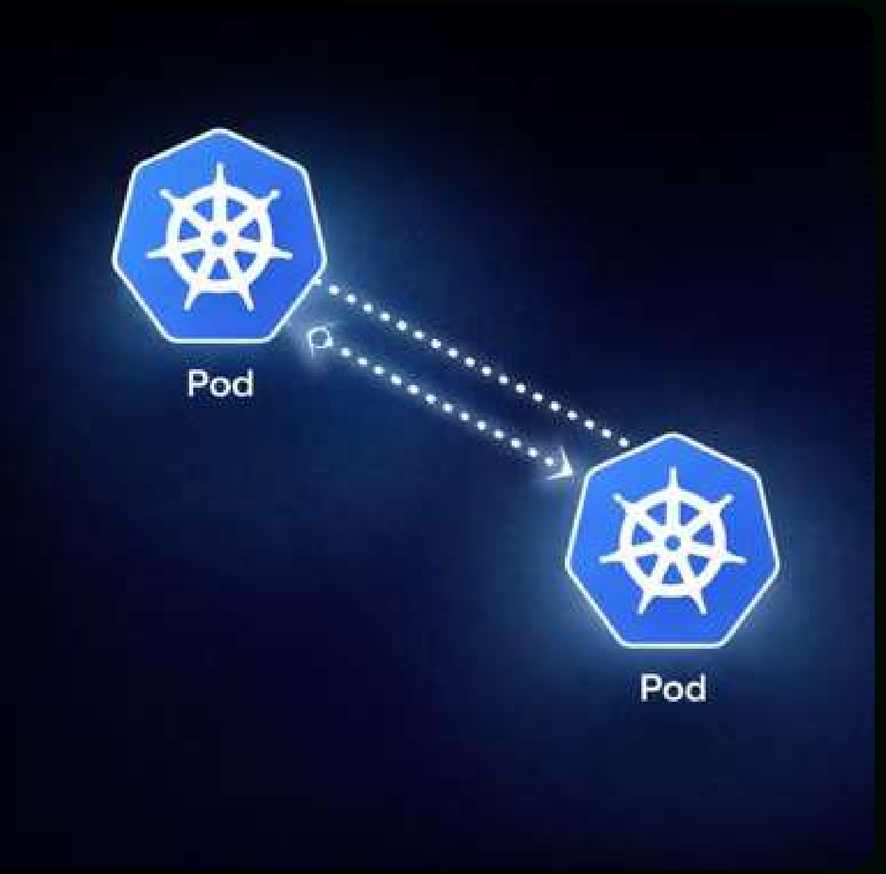
YAML is the declarative configuration format used by Kubernetes to define resources and their desired state. Kubernetes continuously reconciles the cluster to match what is declared in YAML files.

YAML defines intent, Kubernetes handles execution.

2

DECLARATIVE VS IMPERATIVE MODEL

Declarative vs Imperative Model



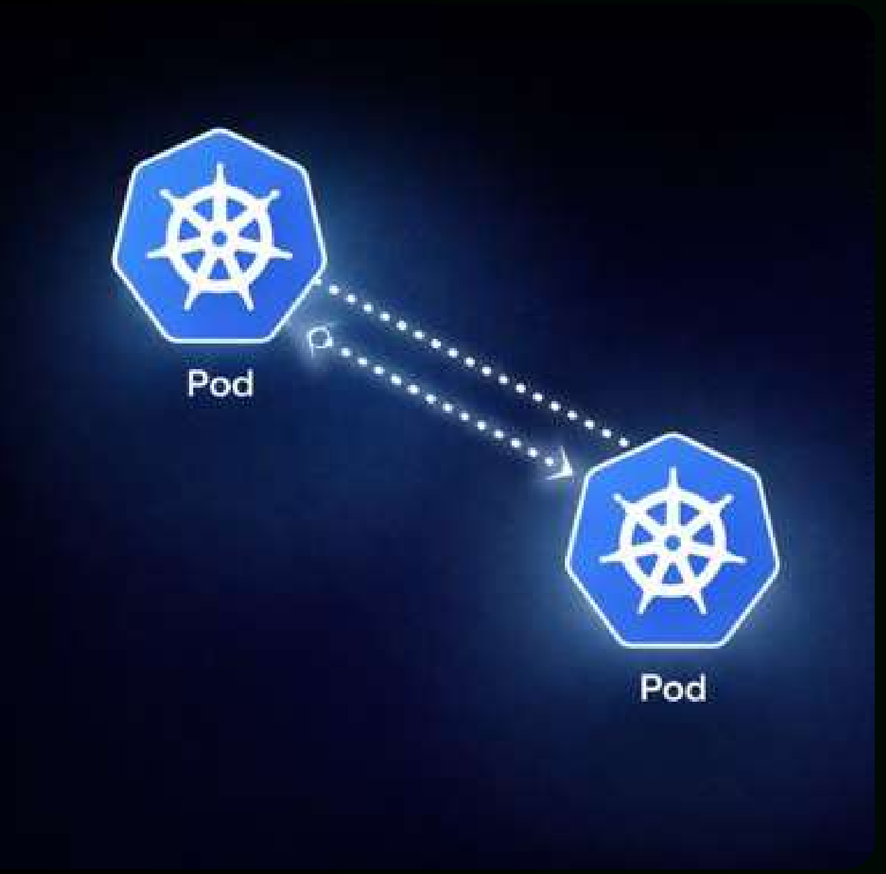
Declarative Model

You define what the desired state should be using YAML. Kubernetes continuously monitors the system and automatically reconciles any difference between the current state and the declared state.

Imperative Model

You explicitly tell the system how to perform each action. Commands describe procedures for creating, updating, or deleting resources manually, and the system does not automatically correct drift unless instructed to.

Example



Declarative Model

kubectl apply -f deployment.yaml

replicas: 3

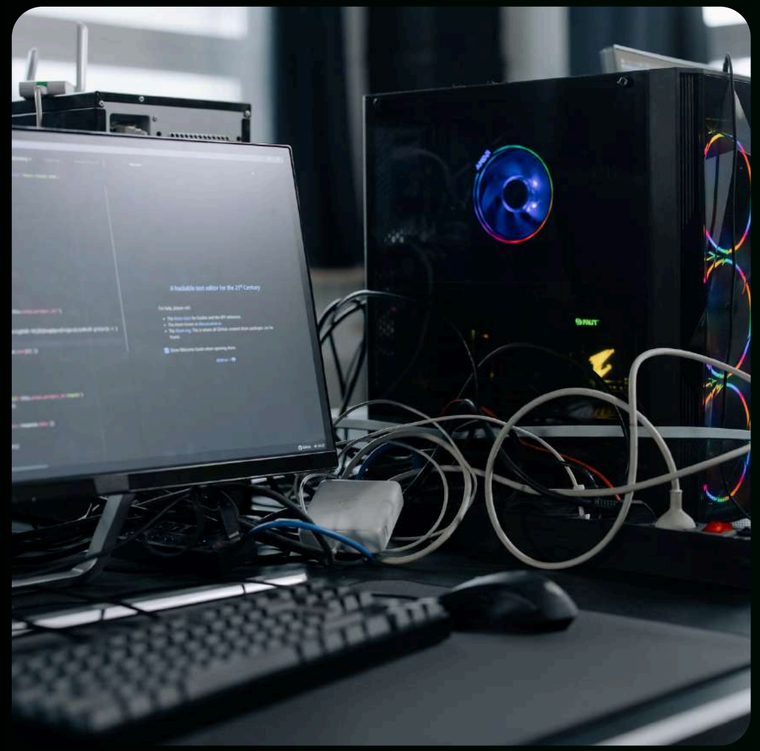
Imperative Model

kubectl run my-app --image=nginx
kubectl expose pod my-app --port=80
kubectl scale pod my-app --replicas=3

Imperative = do this now | Declarative = this should always be true

3

ANATOMY OF A KUBERNETES MANIFEST



Kubernetes Manifest

Think of a Kubernetes manifest as an ID card for a resource.

It tells Kubernetes what this thing is, what it’s called, and how it should behave. The structure is always predictable, so Kubernetes knows exactly how to read it. Once submitted, Kubernetes stores this definition and continuously uses it as the reference for managing the resource’s lifecycle.

The Four Required Headers

apiVersion	**metadata**
kind	**spec**

4

APIVERSION, KIND, & METDATA

In Detail

apiVersion

Tells Kubernetes which API version to use to interpret the resource.

kind

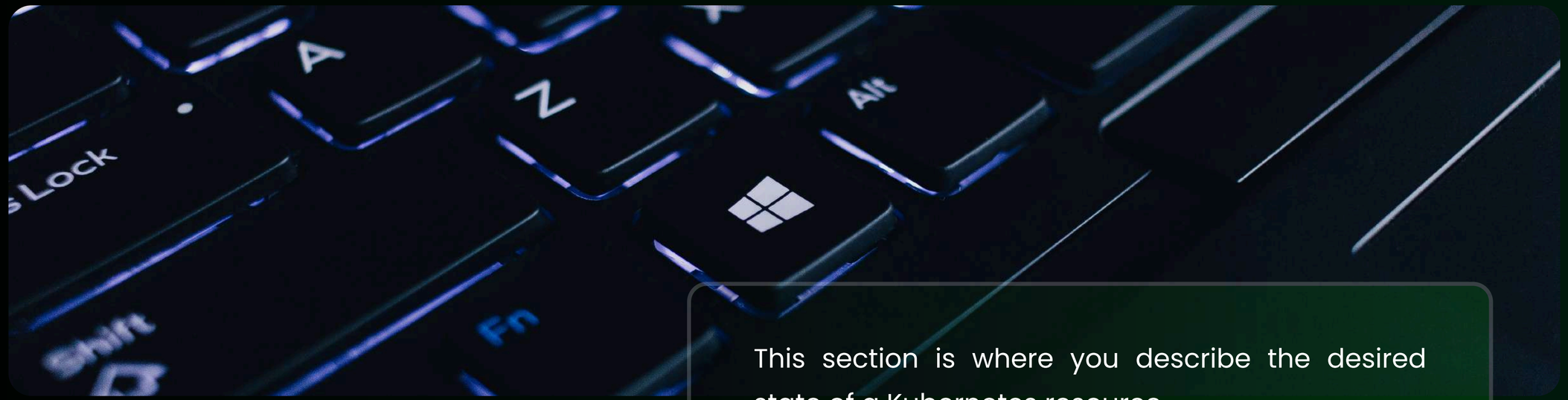
Defines what type of resource is being created, such as a Pod, Deployment, or Service.

metadata

Provides the identity of the resource, including its name, namespace, and labels.

5

SPEC



Spec In Detail

This section is where you describe the desired state of a Kubernetes resource.
It defines how the resource should behave, such as the number of replicas, container images, ports, environment variables, and other runtime settings. Each resource type has its own spec structure, but the goal is always the same: to tell Kubernetes what you want, not how to achieve it.

YAML Example

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: demo
  template:
    metadata:
      labels:
        app: demo
    spec:
      containers:
        - name: app-container
          image: nginx:latest
          ports:
            - containerPort: 80
```

6

COMMON KUBERNETES RESOURCE

Core workload resources

Pod

The smallest deployable unit. Runs one or more containers.

Deployment

Manages replicas, rolling updates, and self-healing for stateless apps.

StatefulSet

Manages stateful workloads with stable identities and storage.

DaemonSet

Ensures a Pod runs on every (or selected) node.

Networking & access resources

Service

Provides stable networking and load balancing for Pods.

Ingress

Manages external HTTP/HTTPS access into the cluster.

Configuration & secrets

ConfigMap

Stores non-sensitive configuration data.

Secret

Stores sensitive data like passwords and tokens.

Configuration & secrets

Job

Runs a task until it completes successfully.

CRON Job

Schedules Jobs to run on a time-based schedule.

7

HOW KUBERNETES APPLIES YAML

How Kubernetes Applies YAML

When you apply a YAML file, Kubernetes stores the desired state in the API server.

Controllers continuously compare the declared state with the current state of the cluster. If a difference is detected, such as a Pod failing or being deleted, Kubernetes automatically takes action to reconcile the state and restore what was declared.

YAML is applied once, reconciliation runs continuously.

8

HANDS-ON YAML LAB

Lab Link

9

KEY TAKEAWAYS & WHAT'S NEXT

Key Takeaways

Kubernetes uses YAML manifests to declare the desired state

Every manifest follows a clear structure:

- apiVersion
- kind
- metadata
- spec

Key Takeaways

YAML is declarative, not command-driven

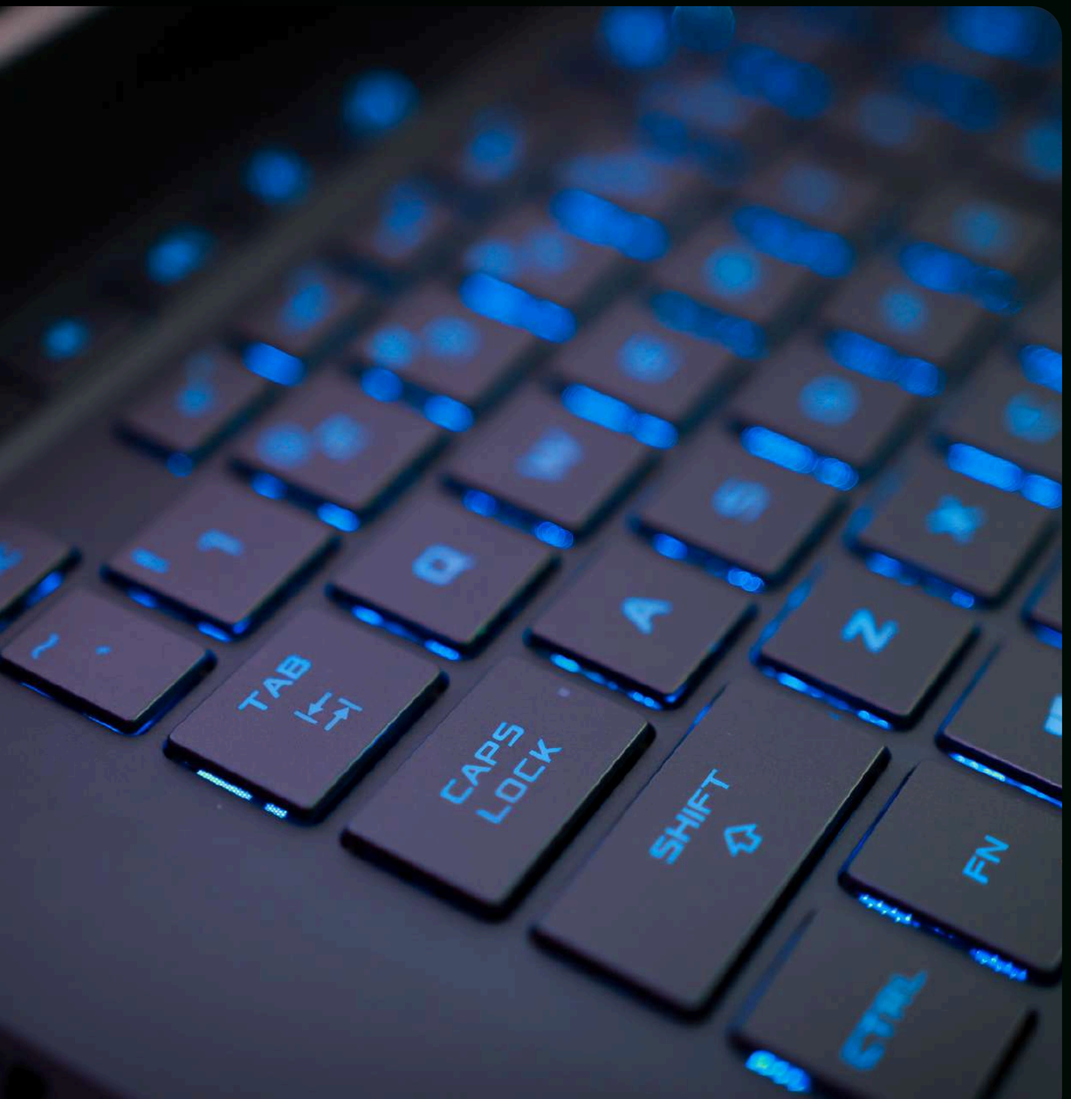
Kubernetes continuously reconciles the actual state to match the manifest

Common resources (Pod, Deployment, Service, ConfigMap, Secret) are just different YAML shapes

Once you understand the structure, reading and writing manifests become predictable

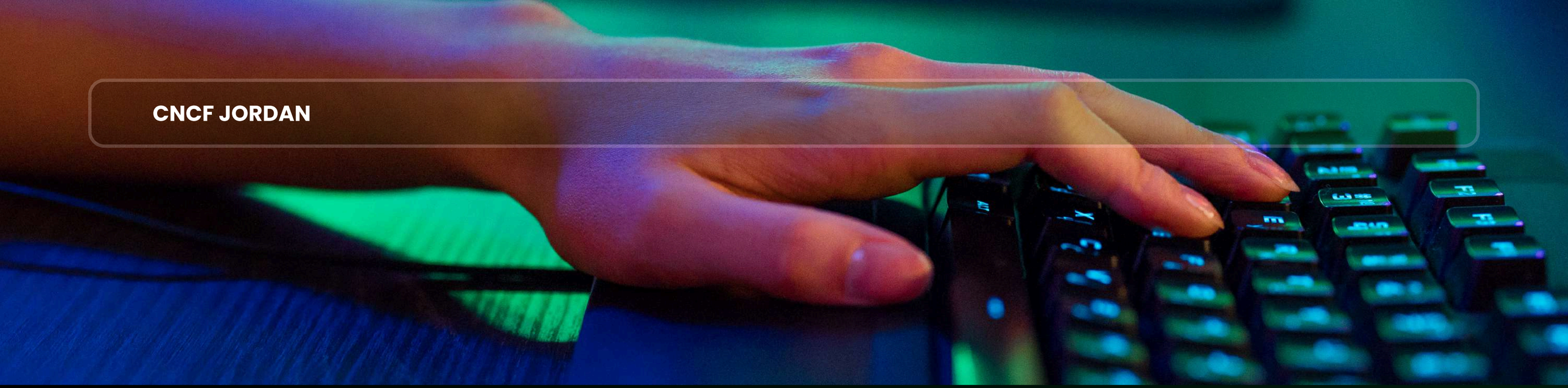
👉 If you can read YAML, you can reason about Kubernetes.

Next Session



Lightning Talk #2
Why Kubernetes Costs More Than You Expect

It will focus on Why Kubernetes Costs More Than You Expect. We'll explore the hidden cost drivers behind Kubernetes, common mistakes teams make when scaling, and why managed clusters don't always mean lower bills. The session will highlight real-world scenarios and practical insights to help teams build cost-aware Kubernetes environments.



THANK YOU

ANY QUESTION